

목차

1장 배포라는 일	2
1.1 배포의 정의	2
1.2 무엇이 달라지는가	3
1.3 되돌릴 수 있는가	4
1.4 배포의 단위	5
1.5 자주 배포하기	6
1.6 한 번의 배포가 지나는 단계	7
1.7 큰 배포의 위험	8
2장 배포 전 점검	9
2.1 점검 항목의 뜻	9
2.2 무엇을 점검표에 넣을까	10
2.3 사람이 하는 확인과 기계가 하는 확인	11
2.4 되돌릴 계획	12
2.5 배포 전 점검표	13
2.6 자료 구조 변경이 걸릴 때	14
2.7 점검표가 형식이 될 때	15
3장 단계적 배포	16
3.1 한꺼번에 바꾸지 않기	16
3.2 일부에게 먼저	17
3.3 새 것과 옛 것을 함께	18
3.4 기능을 켜고 끄기	19
3.5 단계적 배포의 진행	20
3.6 되돌리는 방법	21
3.7 단계가 늘어날 때의 비용	22
4장 상태 확인	23
4.1 잘 올라갔는지 보기	23
4.2 살아 있는지 묻는 방법	24
4.3 준비되었는지와 살아 있는지	25
4.4 배포 직후에 보는 것들	26
4.5 무엇을 보고 성공이라 할까	27
4.6 배포와 무관한 변화	28
4.7 실패를 판정하는 기준	29
5장 배포 뒤 관찰	30
5.1 관찰 준비	30
5.2 언제까지 볼 것인가	31
5.3 지표와 배포를 잇기	32
5.4 천천히 드러나는 문제	33
5.5 사용자가 먼저 알릴 때	34
5.6 되돌린 뒤에 할 일	35
5.7 배포 기록 남기기	36
6장 한 배포를 나눠 진행한 기록	37
6.1 나눠 진행한 배포	37
6.2 첫 단계와 점검	38
6.3 새 코드를 꺼진 채로	39
6.4 일부에게 먼저 컨 결과	40
6.5 멈추고 되돌린 자리	41
6.6 관찰에서 본 것	42
6.7 절차에 더한 것	43
7장 배포 판단	44
7.1 얼마나 나눌 것인가	44
7.2 되돌릴 수 있게 두기	45
7.3 점검표를 손볼 때	46
7.4 배포 점검 항목	47
7.5 관찰을 남기는 순서	48
7.6 사람과 절차	49
7.7 핵심 원칙 정리	50

1.1 배포의 정의

배포는 만든 것을 실제로 쓰이는 자리에 올리는 일이다. 코드가 아무리 잘 짜여 있어도 올라가지 않으면 아무 일도 하지 않고, 올라가는 순간부터 실제 사용자와 실제 자료를 만난다.

그래서 배포는 코드를 만드는 일과 성질이 다르다. 만드는 동안에는 틀려도 되돌리면 그만이지만, 올라간 뒤의 잘못은 사용자가 먼저 겪는다. 되돌리는 데도 시간이 들고, 그 사이에 쌓인 자료는 되돌려지지 않는다.

이 책은 그 차이를 다루는 절차를 정리한다. 올리기 전에 무엇을 확인하고, 어떻게 나눠 올리고, 올린 뒤에 무엇을 보고, 잘못되었을 때 어떻게 되돌릴지가 차례로 나온다. 목표는 배포를 안 틀리게 만드는 것이 아니라 틀렸을 때 빨리 알고 빨리 되돌릴 수 있게 만드는 것이다.

1.2 무엇이 달라지는가

배포가 바뀌는 것은 코드만이 아니다. 설정이 함께 바뀌고, 저장 구조가 바뀌기도 하며, 바깥과 주고받는 약속이 달라지기도 한다. 이 넷은 되돌리는 난이도가 서로 다르다.

코드는 대개 되돌리기 쉽다. 설정도 값을 되돌리면 된다. 그러나 저장 구조는 이미 바뀐 자료가 남아 되돌리기 어렵고, 바깥에 나간 약속은 상대가 이미 그 약속을 쓰고 있으면 거둘 수 없다.

그래서 배포를 준비할 때 이번에 무엇이 바뀌는지 넷으로 나눠 적는다. 코드만 바뀌는 배포와 저장 구조가 함께 바뀌는 배포는 같은 절차로 다루면 안 된다. 이 구분이 뒤에 나올 점검과 단계 나누기의 출발점이다. 무엇이 바뀌는지 적는 일이 배포 준비의 첫 단계다.

1.3 되돌릴 수 있는가

배포 절차의 무게는 되돌릴 수 있는지에 따라 정해진다. 곧바로 되돌릴 수 있는 배포는 가볍게 진행할 수 있고, 되돌릴 수 없는 배포는 올리기 전에 더 많이 확인해야 한다.

되돌릴 수 없게 만드는 것은 대개 셋이다. 자료의 모양을 바꾸는 변경, 바깥으로 내보낸 알림이나 요청, 그리고 한 번 지운 자료다. 이 셋이 없는 배포는 문제가 생겨도 이전 상태로 돌아갈 수 있다.

그래서 배포를 준비할 때 첫 물음은 언제나 이것이다. 잘못되면 어떻게 되돌리는가. 답이 없으면 그 배포는 되돌릴 수 있는 형태로 다시 나눈다. 되돌릴 수 없는 부분을 뒤로 미루고 되돌릴 수 있는 부분을 먼저 올리는 것이 이 책 전체에서 되풀이되는 방법이다.

1.4 배포의 단위

한 번의 배포에 무엇을 담을지는 선택이다. 여러 변경을 모아 한 번에 올릴 수도 있고, 변경 하나씩 따로 올릴 수도 있다. 이 선택이 문제가 생겼을 때의 대응을 정한다.

여럿을 모아 올리면 준비하는 품이 한 번으로 줄지만, 문제가 생겼을 때 어느 변경 때문인지 알 수 없다. 되돌리면 관계없는 변경까지 함께 내려간다. 하나씩 올리면 반대다. 품은 늘지만 원인이 분명하고 되돌리는 범위도 좁다.

그래서 단위는 되돌리기를 기준으로 정한다. 함께 되돌려도 되는 것끼리 묶고, 따로 되돌려야 하는 것은 나눈다. 이 기준으로 보면 서로 무관한 기능 둘을 한 배포에 담는 일은 대개 이득이 없다. 묶는 기준을 정해 두면 무엇을 함께 올릴지 매번 고민하지 않아도 된다.

1.5 자주 배포하기

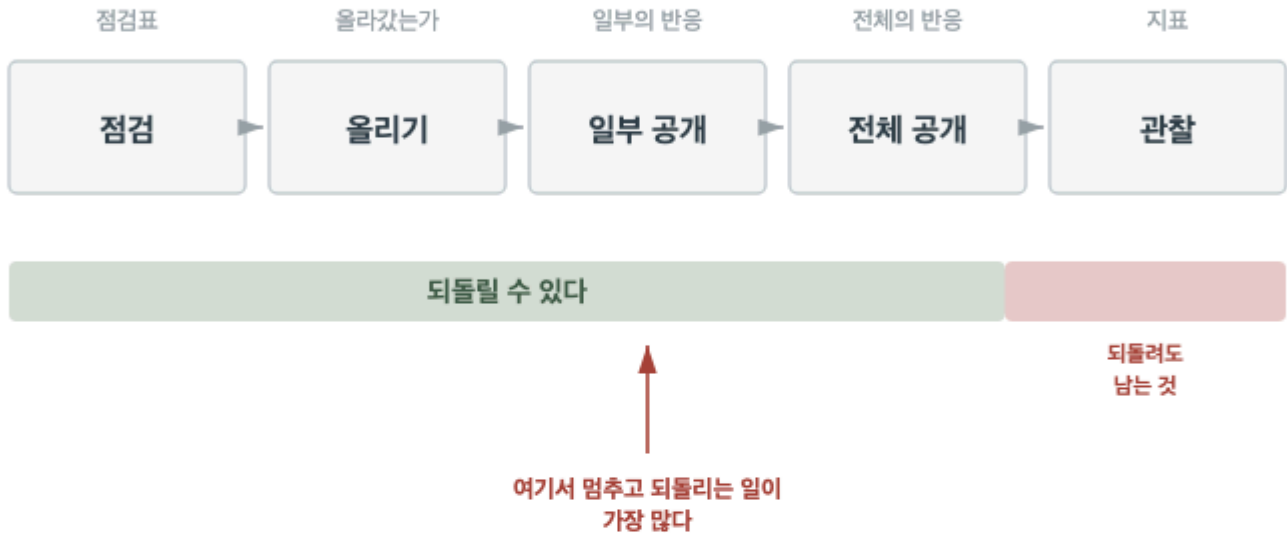
배포를 자주 하면 한 번에 올라가는 변경이 작아진다. 작은 변경은 문제가 생겼을 때 원인이 분명하고, 되돌리기 쉽고, 확인할 것도 적다.

자주 하려면 배포하는 일 자체가 가벼워야 한다. 손으로 하는 단계가 많으면 한 번 하는 데 품이 들어 모아서 하게 되고, 모아서 하면 위험이 커진다. 그래서 배포를 자주 하는 것과 배포를 자동화하는 것은 같은 문제의 두 면이다.

다만 자주 한다고 확인을 건너뛰어도 된다는 뜻은 아니다. 확인을 줄이는 것이 아니라 확인을 빠르게 만드는 것이 방향이다. 앞선 권들에서 다룬 자동 확인이 여기서 배포 절차의 일부가 된다. 가볍게 만드는 일과 자주 하는 일은 서로를 밀어 준다. 확인을 빠르게 만드는 일이 자주 배포하는 일의 조건이다.

한 번의 배포가 지나는 단계

되돌릴 수 있는 자리가 어디까지인지 표시했다



단계를 나누면 멈출 자리가 생긴다

1.7 큰 배포의 위험

여러 주 동안 모은 변경을 한 번에 올리면 위험이 크게 는다. 담긴 변경이 많아 확인할 것이 많고, 문제가 생겼을 때 원인 후보도 그만큼 많다.

더 큰 문제는 되돌리기 어려워진다는 점이다. 큰 배포에는 대개 되돌릴 수 없는 변경이 하나쯤 섞여 있고, 그 하나 때문에 전체를 되돌릴 수 없게 된다. 그러면 앞으로 고치는 수밖에 없는데, 급하게 고치는 일은 또 다른 문제를 만든다.

큰 배포가 만들어지는 이유는 대개 배포가 무거워서다. 한 번 하는 데 반나절이 들면 자주 할 수 없고, 자주 할 수 없으니 모으게 된다. 그래서 큰 배포를 줄이는 방법은 조심하는 것이 아니라 배포를 가볍게 만드는 것이다. 큰 배포를 줄이려면 배포 한 번에 드는 품부터 재어 본다.

2.1 점검 항목의 뜻

배포 전 점검은 올리기 전에 확인할 것을 적어 둔 목록이다. 매번 같은 것을 확인하게 만들어, 급할 때 빠뜨리는 것을 줄이는 장치다.

점검표가 필요한 이유는 배포가 대개 급하게 진행되기 때문이다. 급하면 기억에 의존하게 되고, 기억은 그날의 상태에 따라 다르다. 적어 두면 그날의 상태와 무관하게 같은 것이 확인된다.

점검표의 항목은 실제로 겪은 문제에서 온다. 일반적인 좋은 항목을 모아 만든 목록은 길기만 하고 지켜지지 않는다. 이번 배포에서 무엇이 잘못될 수 있는지 물어 나온 답이 다음 배포의 점검 항목이 된다. 목록이 있으면 급한 날에도 확인의 내용이 달라지지 않는다. 겪은 문제에서 나온 항목만 남기면 목록이 저절로 짧아진다.

2.2 무엇을 점검표에 넣을까

점검표는 짧아야 지켜진다. 그래서 넣을 항목을 고르는 기준이 필요하다. 기준은 빠뜨렸을 때의 손해가 크고 빠뜨리기 쉬운 것이다.

이 기준으로 보면 대개 넷이 남는다. 저장 구조 변경이 함께 있는지, 설정이 환경마다 갖춰져 있는지, 되돌릴 방법이 준비되어 있는지, 그리고 이번 변경이 바깥과의 약속을 바꾸는지다. 넷 다 빠뜨리면 되돌리기 어려운 상황을 만든다.

반대로 빼도 되는 항목은 이미 자동으로 확인되는 것들이다. 확인이 자동으로 도는데 점검표에도 적어 두면 사람이 같은 것을 두 번 보게 되고, 두 번 보는 목록은 곧 형식이 된다. 사람의 점검표에는 기계가 못 보는 것만 남긴다. 항목이 적을수록 실제로 읽히고 지켜진다.

2.3 사람이 하는 확인과 기계가 하는 확인

확인에는 기계가 잘하는 것과 사람이 잘하는 것이 있다. 코드가 도는지, 확인이 통과하는지, 설정 값이 채워져 있는지는 기계가 본다. 이번 변경이 지금 올려도 되는 변경인지, 되돌릴 계획이 실제로 말이 되는지는 사람이 본다.

나누는 기준은 판단이 필요한가다. 정해진 조건과 견주는 일은 기계가 하고, 상황을 보고 정하는 일은 사람이 한다. 사람이 매번 같은 조건을 눈으로 확인하고 있다면 그 일은 기계로 옮긴다.

옮기는 일은 한 번에 되지 않으므로 점검표에 표시를 둔다. 어느 항목이 기계 확인이고 어느 항목이 사람 확인인지 적어 두면, 기계로 옮길 후보가 목록에서 저절로 드러난다. 옮길 수 있는 항목을 옮기면 사람의 목록이 짧아진다.

2.4 되돌릴 계획

되돌릴 계획은 배포 전에 적는다. 문제가 생긴 뒤에 정하면 급한 상황에서 판단하게 되고, 급한 판단은 대개 상황을 더 나쁘게 만든다.

계획에는 셋을 적는다. 무엇을 보고 되돌릴지, 어떻게 되돌릴지, 되돌린 뒤에 어떤 상태가 되는지다. 특히 셋째가 자주 빠진다. 되돌렸는데 자료가 새 모양으로 남아 있으면 옛 코드가 그 자료를 다루지 못한다.

계획을 적다 보면 되돌릴 수 없는 배포라는 것이 드러나기도 한다. 그때는 배포를 나눈다. 되돌릴 수 없는 부분을 따로 떼어 먼저 올리고, 그 부분이 안정된 뒤에 나머지를 올리면 각 단계가 되돌릴 수 있는 상태가 된다. 계획을 적는 일이 곧 이 배포를 나눌지 정하는 일이 된다.

배포 전 점검표

기계가 보는 것과 사람이 보는 것을 나눠 적는다

점검 항목	확인 주체	
확인이 모두 통과했는가	기계	사람이 다시 보지 않는다
설정이 환경마다 갖춰졌는가	기계	
저장 구조 변경이 있는가	사람	
되돌릴 방법이 준비되었는가	사람	
바깥과의 약속이 바뀌는가	사람	
올려도 되는 때인가	사람	

항목이 늘면 지켜지지 않는다

사람의 목록에는 판단이 필요한 것만 남긴다

2.6 자료 구조 변경이 걸릴 때

저장 구조가 바뀌는 배포는 다른 배포와 다르게 다룬다. 코드는 되돌릴 수 있지만 이미 바뀐 자료는 되돌릴 수 없기 때문이다.

그래서 순서를 나눈다. 먼저 새 구조를 옛 코드도 견딜 수 있는 형태로 더한다. 항목을 더할 때는 없어도 되게 두고, 지울 때는 곧바로 지우지 않고 쓰지 않는 상태로 둔다. 그다음에 새 코드를 올리고, 새 코드가 안정된 뒤에 옛 항목을 지운다.

이 순서를 지키면 각 단계에서 되돌릴 수 있다. 한 번에 바꾸면 그 순간부터 되돌릴 수 없는 상태가 되고, 문제가 생기면 앞으로 고치는 수밖에 없다. 저장 구조 변경은 대개 이 순서를 지키는 것만으로 위험이 크게 준다. 순서를 지키면 각 단계가 스스로 되돌릴 수 있는 상태로 남는다.

2.7 점검표가 형식이 될 때

점검표는 시간이 지나면 형식이 되기 쉽다. 항목이 늘고, 대부분이 늘 통과하고, 확인하지 않아도 문제가 없던 경험이 쌓이면 사람들은 읽지 않고 표시만 한다.

이렇게 되는 원인은 대개 항목의 성격에 있다. 늘 통과하는 항목, 확인하는 방법이 적혀 있지 않은 항목, 이 배포와 상관없는 항목이 섞여 있으면 목록 전체가 신뢰를 잃는다.

그래서 점검표도 이따금 정리한다. 최근 배포에서 실제로 걸린 적이 있는 항목만 남기고, 한 번도 걸리지 않은 항목은 기계 확인으로 옮기거나 지운다. 짧고 실제로 걸리는 목록이 길고 늘 통과하는 목록보다 안전하다. 짧고 실제로 걸리는 목록만 신뢰를 유지한다. 표시만 하는 목록은 없는 목록보다 위험하다.

3.1 한꺼번에 바꾸지 않기

단계적 배포는 새 것을 모든 사용자에게 한꺼번에 열지 않고 조금씩 넓혀 가는 방법이다. 일부에게 먼저 열고 문제가 없으면 범위를 넓히며, 문제가 있으면 그 자리에서 멈춘다.

이 방법의 값은 실제 사용자와 실제 자료에서만 드러나는 문제를 작은 범위에서 만난다는 데 있다. 아무리 확인해도 실제 환경의 조합은 다 만들 수 없으므로, 마지막 확인은 실제에서 하되 범위를 좁히는 것이다.

나누는 방법은 여럿이다. 사용자의 일부에게만 열기, 서버를 나눠 일부에만 새 것을 올리기, 기능을 켜고 끄는 값으로 조절하기다. 무엇을 쓰든 원칙은 같다. 문제가 났을 때 영향을 받는 범위가 좁고, 멈추는 일이 쉬워야 한다. 무엇으로 나누든 멈출 수 있는 자리를 만드는 것이 목적이다.

3.2 일부에게 먼저

일부에게 먼저 연다면 그 일부를 어떻게 고를지 정해야 한다. 아무나 고르면 문제가 드러나지 않을 수 있고, 특정 무리만 고르면 그 무리에만 있는 조건이 결과를 왜곡한다.

실용적인 방법은 두 가지를 섞는 것이다. 먼저 내부 사용자나 시험에 동의한 사용자에게 열어 눈에 띄는 문제를 걸러 내고, 그다음에 전체에서 고르게 뽑은 일부에게 연다. 앞쪽은 빠르게 알 수 있고 뒤쪽은 실제 분포를 반영한다.

범위를 넓히는 속도도 미리 정한다. 문제가 없으면 언제 얼마나 넓힐지 적어 두면 판단을 그때그때 하지 않아도 된다. 넓히는 중에도 언제든지 멈출 수 있어야 하므로, 되돌리는 방법은 각 단계에서 유지된다. 넓히는 속도를 미리 정해 두면 판단이 그날의 분위기에 흔들리지 않는다.

3.3 새 것과 옛 것을 함께

단계적으로 배포하면 새 코드와 옛 코드가 한동안 함께 돈다. 이 기간에는 두 벌이 같은 자료를 다루므로, 자료의 모양이 양쪽에서 모두 읽히고 쓰여야 한다.

그래서 자료 변경은 늘 두 벌이 함께 견디는 형태로 만든다. 항목을 더할 때는 없어도 되게 하고, 이름을 바꿀 때는 둘을 함께 두고, 지울 때는 아무도 쓰지 않는 것을 확인한 뒤에 지운다. 이 규칙을 지키지 않으면 배포 도중에 오류가 나거나 자료가 어긋난다.

바깥과의 약속도 같다. 새 코드가 보내는 요청을 옛 상대가 받을 수 있어야 하고, 반대도 되어야 한다. 병행 기간은 짧을수록 좋지만 없을 수는 없으므로, 그 기간을 전제로 설계하는 편이 안전하다. 병행 기간을 전제로 만들면 배포 도중의 오류가 크게 준다.

3.4 기능을 켜고 끄기

코드를 올리는 일과 기능을 여는 일을 나눌 수 있다. 새 기능을 꺼진 상태로 올려 두고, 값 하나로 여는 방식이다. 이렇게 하면 배포와 공개가 분리된다.

이 방식의 이득은 셋이다. 문제가 생겼을 때 다시 배포하지 않고 값만 되돌리면 되고, 일부에게만 여는 일이 쉬워지며, 배포 자체를 작게 유지할 수 있다.

대신 값이 늘어나는 부담이 있다. 켜고 끄는 값이 쌓이면 조합이 늘어 무엇이 켜진 상태인지 알기 어려워지고, 꺼진 채로 남은 코드가 낡아 간다. 그래서 값을 더할 때 언제 지울지 함께 정한다. 전환 값은 임시 장치이지 설정이 아니다. 전환 값에는 지울 시점을 함께 적어 둔다. 값이 늘어난 만큼 지우는 일도 함께 계획한다.

단계적 배포의 진행

넓히는 중에도 언제든지 멈출 수 있어야 한다



넓히는 길과 되돌리는 길이 모두 있어야 한다

3.6 되돌리는 방법

되돌리는 방법은 배포 방식에 따라 다르다. 기능 전환 값으로 열었다면 값을 되돌리면 되고, 서버를 나눠 올렸다면 옛 것을 다시 올리면 되며, 전체를 바꿨다면 이전 판을 다시 올려야 한다. 앞의 둘은 빠르고 셋째는 느리다.

어느 방법이든 확인할 것은 같다. 되돌린 뒤 자료가 옛 코드로 다룰 수 있는 상태인지, 그동안 쌓인 자료가 어떻게 되는지, 되돌렸다는 사실이 관련된 사람들에게 알려지는지다.

되돌리기는 실패가 아니라 준비된 선택이다. 되돌린 뒤 원인을 찾아 고치고 다시 올리는 것이 정상 경로다. 되돌리기를 실패로 여기면 되돌려야 할 상황에서 버티게 되고, 버티는 동안 손해가 커진다. 되돌리기가 쉬울수록 판단이 빨라지고 손해가 작아진다.

3.7 단계가 늘어날 때의 비용

단계를 나누면 안전해지지만 비용이 있다. 각 단계마다 기다리고 확인해야 하므로 배포가 끝나기까지 오래 걸린다. 병행 기간이 길어지면 두 벌을 함께 다루는 코드도 오래 남는다.

그래서 모든 배포를 잘게 나누지는 않는다. 되돌리기 쉽고 영향이 작은 변경은 한 번에 올리고, 되돌리기 어렵거나 영향이 큰 변경만 나눈다. 나누는 정도를 변경의 성질에 맞추는 것이다.

판단 기준은 앞 장과 같다. 잘못되면 무엇이 얼마나 나빠지는가. 나빠지는 것이 작으면 나누지 않고, 크면 나눈다. 모든 배포에 같은 절차를 쓰면 작은 배포가 무거워지고, 무거워진 배포는 다시 모아서 하게 된다. 변경의 성질에 맞춘 절차가 모든 배포에 같은 절차보다 안전하다.

4.1 잘 올라갔는지 보기

배포가 끝났다는 것과 잘 동작한다는 것은 다르다. 파일이 올라갔고 프로그램이 시작되었다는 사실만으로 새 코드가 제대로 도는지 알 수 없다.

그래서 배포 직후에 세 가지를 본다. 새 판이 실제로 올라갔는지, 프로그램이 요청을 받을 준비가 되었는지, 그리고 대표적인 흐름 하나가 끝까지 되는지다. 셋째가 가장 중요한데 가장 자주 생략된다.

확인인 사람이 눈으로 보는 대신 자동으로 돌게 만든다. 배포한 사람이 화면을 열어 확인하는 방식은 그 사람이 없으면 되지 않고, 매번 다르게 확인된다. 자동으로 도는 짧은 확인 하나가 눈으로 보는 열 번보다 낫다. 확인이 자동으로 돌면 누가 배포해도 같은 확인이 이루어진다. 대표 흐름 하나를 끝까지 돌려 보는 확인이 가장 값이 크다.

4.2 살아 있는지 묻는 방법

프로그램이 살아 있는지 묻는 방법은 그 프로그램이 답하는 자리를 두는 것이다. 정해진 자리로 물으면 지금 상태를 짧게 답하고, 그 답으로 올릴지 내릴지 판단한다.

답하는 내용은 짧을수록 좋다. 여기에 무거운 확인을 담으면 물을 때마다 부담이 되고, 다른 시스템의 상태까지 담으면 상대가 느릴 때 이쪽이 죽은 것으로 판정된다. 이쪽이 답할 수 있는 것만 답한다.

그래서 상태를 묻는 자리는 두 종류로 나눠 두는 경우가 많다. 지금 살아 있는지만 답하는 자리와, 요청을 받을 준비가 되었는지 답하는 자리다. 다음 절이 이 구분을 다룬다.답이 가벼울수록 자주 물을 수 있고 판단도 빨라진다.이쪽이 답할 수 있는 것만 답하게 두는 것이 원칙이다.

4.3 준비되었는지와 살아 있는지

살아 있는지와 준비되었는지는 다른 물음이다. 프로그램이 막 시작해 아직 준비 중일 때는 살아 있지만 요청을 받을 수는 없다. 반대로 오래 돌아 내부가 망가지면 요청은 받지만 제대로 답하지 못한다.

두 물음을 나누면 각각에 맞는 대응을 할 수 있다. 준비되지 않았다면 기다리고, 살아 있지 않다면 다시 시작한다. 하나로 합치면 준비 중인 프로그램을 계속 다시 시작해 영영 준비되지 않는 상황이 생긴다.

배포에서 이 구분이 중요한 것은 새 판을 올릴 때다. 새 판이 준비될 때까지 옛 판을 내리지 않아야 요청이 끊기지 않는다. 준비되었는지 묻는 자리가 없으면 이 조절을 할 수 없어 배포할 때마다 짧게 끊긴다. 두 물음을 나눈 시스템만 끊김 없는 교체가 가능하다.

배포 직후에 보는 것들

올라간 것과 도는 것은 다른 물음이다



보는 순서가 곧 멈출 자리의 순서다

4.5 무엇을 보고 성공이라 할까

배포의 성공을 판정하려면 기준이 필요하다. 기준이 없으면 아무 일도 일어나지 않은 것을 성공으로 여기게 되고, 실제로는 조용히 나빠지고 있어도 알지 못한다.

기준은 배포 전에 정한다. 오류 비율이 평소 범위 안에 있고, 응답 시간이 평소와 비슷하고, 이번 변경이 겨누는 지표가 예상대로 움직이면 성공이다. 셋째가 있어야 이 배포가 무엇을 하려던 것인지 확인된다.

기준에는 볼 시간도 함께 적는다. 십 분 안에 판단할 것인지 하루를 볼 것인지에 따라 다음 단계로 넘어가는 시점이 달라진다. 시간을 적지 않으면 아무도 끝났다고 말하지 못하고 배포가 계속 열린 채로 남는다. 기준과 시간을 함께 적어야 배포가 끝났다고 말할 수 있다.

4.6 배포와 무관한 변화

배포 직후에 지표가 흔들렸다고 그것이 배포 때문이라고 단정할 수 없다. 같은 시각에 다른 일이 일어났을 수 있고, 원래 그 시간대에 그런 움직임이 있었을 수도 있다.

가르는 방법은 견주는 것이다. 지난 같은 요일 같은 시각과 견주고, 아직 새 판이 가지 않은 쪽과 견준다. 단계적 배포를 하고 있으면 뒤쪽 비교가 가능해지므로, 나눠 올리는 일은 판정에도 도움이 된다.

그래도 확실하지 않으면 되돌려 본다. 되돌렸을 때 지표가 돌아오면 배포 때문이고, 그대로면 다른 이유다. 되돌리기가 쉬운 상태로 만들어 두면 이 판단을 빠르게 할 수 있다는 것이 여기서도 이득이 된다. 견줄 대상이 있으면 판단이 짐작에서 확인으로 바뀐다.

4.7 실패를 판정하는 기준

성공 기준만큼 실패 기준도 미리 정한다. 오류가 얼마를 넘으면 멈추고, 응답이 얼마를 넘으면 멈춘다는 식이다. 정해 두지 않으면 판단이 그 자리의 분위기에 좌우된다.

실패 기준은 성공 기준보다 낮게 잡는다. 애매한 구간에서는 멈추는 쪽을 고르는 것이 안전하기 때문이다. 멈춰서 잃는 것은 시간이고, 버텨서 잃는 것은 사용자의 신뢰와 자료다.

기준을 넘었을 때 누가 결정하는지도 정해 둔다. 결정하는 사람이 정해져 있지 않으면 모두가 서로를 기다리고, 그 사이에 시간이 간다. 미리 정한 기준과 미리 정한 결정권이 급한 상황에서 판단을 대신해 준다. 기준과 결정권을 미리 정해 두면 멈추는 데 걸리는 시간이 준다. 애매할 때 멈추는 쪽을 고르는 것이 대체로 싸다.

5.1 관찰 준비

배포 뒤에 무엇을 볼지는 배포 전에 정한다. 올린 뒤에 지표를 만들기 시작하면 비교할 이전 값이 없어 아무 판단도 할 수 없다.

준비할 것은 셋이다. 이번 변경이 움직일 지표, 그 지표의 평소 값, 그리고 그것을 볼 자리다. 평소 값이 특히 중요하다. 지금 값이 높은지 낮은지는 평소를 알아야 말할 수 있다.

새 기능을 넣는 배포라면 그 기능이 쓰이는지 셀 방법도 함께 넣는다. 기능은 올라갔는데 아무도 쓰지 않는 상태를 아는 것과 모르는 것은 다르다. 배포의 성공은 오류가 없는 것만이 아니라 하려던 일이 일어나는 것이기도 하다. 볼 것을 미리 갖추는 일이 배포 준비의 절반이다. 평소 값이 없으면 지금 값이 높은지 낮은지 말할 수 없다.

5.2 언제까지 볼 것인가

관찰에는 끝이 있어야 한다. 끝을 정하지 않으면 배포가 언제 끝났는지 아무도 말하지 못하고, 며칠 뒤에 생긴 문제까지 그 배포 탓으로 남는다.

기간은 변경의 성질에 따라 정한다. 곧바로 드러나는 변경은 몇십 분이면 되고, 하루 주기로 도는 처리에 걸린 변경은 최소 하루를 본다. 월말에만 도는 처리가 걸렸다면 그때까지가 관찰 기간이다.

관찰이 끝나면 끝났다고 적는다. 이 표시가 있어야 다음 배포를 시작할 수 있고, 문제가 생겼을 때 어느 배포까지 확인되었는지 알 수 있다. 끝을 적지 않은 배포가 여럿 겹치면 원인을 좁힐 수 없게 된다. 끝을 적어야 다음 배포를 안심하고 시작할 수 있다. 관찰이 겹치면 어느 배포까지 확인되었는지 알 수 없게 된다.

5.3 지표와 배포를 잇기

지표를 볼 때 배포 시점이 함께 표시되어 있으면 판단이 빨라진다. 지표가 꺾인 자리에 배포 표시가 있으면 후보가 하나로 좁혀지고, 없으면 다른 이유를 찾게 된다.

표시에는 시각만이 아니라 무엇이 올라갔는지도 함께 적는다. 하루에 여러 번 배포하는 경우 시각만 있으면 어느 배포인지 알 수 없다. 판의 이름과 담긴 변경의 요약이 있으면 그 자리에서 짐작할 수 있다.

이 표시는 배포 절차에서 자동으로 남기게 만든다. 사람이 손으로 적으면 급할 때 빠지고, 급할 때의 배포가 대개 문제를 만드는 배포다. 자동으로 남는 표시 하나가 나중의 조사 시간을 크게 줄인다. 표시 하나가 조사 시간을 몇 시간씩 줄이기도 한다. 표시가 없으면 같은 조사를 매번 처음부터 하게 된다.

5.4 천천히 드러나는 문제

배포 직후에는 아무 문제가 없다가 며칠 뒤에 드러나는 문제가 있다. 자료가 쌓이면서 느려지는 경우, 주기적으로 도는 처리에서만 나는 경우, 저장 공간이나 연결이 조금씩 새는 경우다.

이런 문제는 배포와 시간이 떨어져 있어 연결 짓기 어렵다. 그래서 앞 절의 배포 표시가 특히 중요해진다. 지표를 며칠 범위로 놓고 보았을 때 표시가 함께 보이면 짐작이 가능해진다.

관찰 기간을 그 주기에 맞추는 것도 방법이다. 하루 한 번 도는 처리에 걸린 변경은 적어도 한 번은 돌아보는 것을 확인한 뒤에 끝낸다. 다 볼 수는 없으므로, 무엇을 못 보고 넘어가는지 알고 넘어가는 것이 목표다. 못 보는 자리를 알고 넘어가는 것과 모르고 넘어가는 것은 다르다.

5.5 사용자가 먼저 알릴 때

모든 문제가 지표에 나타나지는 않는다. 특정 조건에서만 나는 오류, 화면의 일부만 깨지는 문제, 값이 틀렸지만 오류는 나지 않는 경우는 지표가 조용하다.

그래서 사용자가 알리는 통로를 관찰의 일부로 본다. 배포 직후에 들어온 신고는 그 배포와 관련이 있을 가능성이 높으므로, 배포 기간에는 신고를 평소보다 빨리 살핀다.

신고가 들어오면 그 내용을 다음 관찰 항목으로 옮긴다. 같은 문제가 지표로 잡히도록 만들면 다음 번에는 사람이 알리기 전에 알 수 있다. 사용자 신고는 관찰이 빠뜨린 자리를 알려 주는 목록이기도 하다. 신고를 지표로 옮기면 다음에는 사람보다 먼저 알 수 있다. 배포 기간에는 신고를 평소보다 자주 살핀다.

5.6 되돌린 뒤에 할 일

되돌린 것으로 끝이 아니다. 되돌린 뒤에는 세 가지가 남는다. 그 사이에 쌓인 자료를 어떻게 할지, 원인이 무엇인지, 그리고 다시 올릴 때 무엇을 다르게 할지다.

자료는 가장 급하다. 새 코드가 만든 자료가 옛 코드에서 잘못 읽히면 되돌린 뒤에 새 문제가 생긴다. 되돌리기 전에 이 부분을 확인하고, 필요하면 자료를 정리하는 작업을 함께 계획한다.

원인을 찾는 일은 되돌린 뒤에 여유를 가지고 한다. 되돌렸으므로 급하지 않고, 급하지 않으면 제대로 볼 수 있다. 되돌리기의 값은 문제를 없애는 데 있지 않고 문제를 급하지 않게 만드는 데 있다. 되돌린 뒤의 여유가 원인을 제대로 찾게 해 준다. 자료 정리는 되돌리기 전에 계획해 두어야 한다.

5.7 배포 기록 남기기

배포 기록은 나중에 가장 자주 찾게 되는 자료다. 언제 무엇이 올라갔는지, 누가 결정했는지, 관찰이 언제 끝났는지, 되돌린 적이 있는지가 여기 담긴다.

기록의 첫 쓸모는 원인 좁히기다. 문제가 생긴 시점 앞뒤로 어떤 배포가 있었는지 알면 후보가 크게 준다. 둘째 쓸모는 절차를 고치는 근거다. 되돌린 배포가 왜 되돌아갔는지 모아 보면 점검표에 더할 항목이 나온다.

기록은 자동으로 남는 부분과 사람이 적는 부분을 나눈다. 시각과 판과 담긴 변경은 자동으로 남기고, 왜 이 시점에 올렸는지와 관찰에서 무엇을 보았는지는 사람이 적는다. 사람이 적을 것이 적을수록 기록이 실제로 남는다. 기록이 쌓이면 절차를 고칠 근거가 저절로 모인다.

6.1 나눠 진행한 배포

이 장은 배포 하나를 나눠 진행한 기록이다. 변경은 주문 목록의 조회 방식을 바꾸는 것이었고, 코드와 저장 구조가 함께 바뀌었다. 항목 하나를 더하고 그 항목으로 찾도록 바꾸는 변경이다.

먼저 무엇이 바뀌는지 넷으로 나눠 적었다. 코드가 바뀌고, 저장 구조가 바뀌고, 설정이 하나 늘고, 바깥과의 약속은 그대로였다. 이 가운데 되돌릴 수 없는 것은 저장 구조였다.

그래서 한 번에 올리지 않기로 했다. 되돌릴 수 있는 상태를 유지하면서 세 번에 나눠 올리는 계획을 세웠고, 각 단계마다 무엇을 보고 다음으로 넘어갈지 정했다. 이어지는 절은 그 순서를 따라간다. 되돌릴 수 없는 부분이 하나라도 있으면 나누는 쪽이 안전하다.

6.2 첫 단계와 점검

첫 단계는 저장 구조만 바꾸는 것이었다. 새 항목을 더하되 없어도 되게 두었고, 옛 코드는 그 항목을 모르는 채로 그대로 돌아왔다.

점검표에서 이 단계에 걸린 항목은 둘이었다. 저장 구조 변경이 있는가와 되돌릴 방법이 준비되었는가다. 되돌리는 방법은 더한 항목을 지우는 것인데, 이 시점에는 아무도 그 항목을 쓰지 않으므로 안전했다.

올린 뒤에는 자료가 정상으로 읽히는지와 처리 시간이 늘지 않았는지를 보았다. 항목을 더하는 것만으로 느려지는 경우가 있어 확인이 필요했다. 두 시간을 보고 이상이 없어 다음 단계로 넘어갔다. 이 단계는 되돌리기가 가장 쉬워 부담 없이 진행할 수 있었다. 확인에 두 시간을 쓴 것이 이후 단계의 판단을 가볍게 했다.

6.3 새 코드를 꺼진 채로

둘째 단계는 새 조회 방식을 담은 코드를 올리되 켜지 않는 것이었다. 전환 값을 하나 두고 꺼진 상태로 올렸다.

이 단계에서 확인한 것은 새 코드가 올라간 뒤에도 옛 동작이 그대로인가였다. 꺼져 있으므로 달라질 것이 없어야 하는데, 실제로는 목록의 정렬이 미세하게 달라진 자리가 하나 있었다. 전환 값과 무관한 부분이 함께 바뀌어 있었던 것이다.

이 자리를 고치고 다시 올렸다. 꺼진 채로 올리는 단계가 없었다면 이 문제는 새 방식을 켜는 순간 함께 드러나 원인을 가리기 어려웠을 것이다. 단계를 나눈 이득이 처음 드러난 자리였다. 꺼진 채로 올리는 단계가 원인을 하나씩 떼어 놓았다. 무관한 변경이 섞여 있었다는 사실을 이 단계에서 알았다.

6.4 일부에게 먼저 켜 결과

셋째 단계에서 전환 값을 켜다. 먼저 내부 사용자에게만 켜고 눈에 띄는 문제가 없어 전체의 일부에게 고르게 켜다.

성공 기준은 배포 전에 정한 대로 셋이었다. 오류 비율이 평소 범위 안일 것, 목록 화면의 응답 시간이 줄어든 것, 그리고 조회 방식이 실제로 새 항목을 쓰고 있을 것이다. 셋째는 새로 넣은 계수로 확인했다.

일부에게 켜 뒤 응답 시간은 기대대로 줄었지만 오류가 조금 늘었다. 늘어난 오류를 보니 항목이 아직 채워지지 않은 옛 자료에서만 나고 있었다. 여기서 멈추고 되돌린 뒤 옛 자료를 채우는 작업을 먼저 했다. 일부에게만 켜기 때문에 영향을 받은 범위도 좁았다. 성공 기준 셋 가운데 둘만 맞았다는 것이 판단의 근거였다.

6.5 멈추고 되돌린 자리

오류 비율이 미리 정한 실패 기준을 넘었다. 판단은 빨랐다. 기준이 적혀 있었고 결정할 사람도 정해져 있었기 때문이다.

되돌리기는 전환 값을 끄는 것으로 끝났다. 코드를 다시 배포할 필요가 없어 몇 분 만에 옛 동작으로 돌아갔다. 저장 구조는 그대로 두었는데, 항목이 더해져 있어도 옛 코드가 무시하므로 문제가 없었다.

되돌린 뒤 옛 자료의 항목을 채우는 작업을 따로 진행했다. 이 작업은 며칠에 걸쳐 조금씩 돌렸고 끝난 뒤에 다시 켜다. 두 번째로 켜을 때는 오류가 늘지 않았고 응답 시간만 줄었다. 멈추는 결정이 빨랐던 것이 이 배포에서 가장 잘한 일이었다. 옛 자료를 채우는 작업을 먼저 한 뒤에 다시 켜다. 기준이 적혀 있었기에 논의 없이 결정할 수 있었다.

6.6 관찰에서 본 것

전체에 컨 뒤 관찰 기간을 사흘로 잡았다. 이 화면과 관련된 처리 가운데 하루에 한 번 도는 것이 있었기 때문이다.

첫날은 문제가 없었고 둘째 날 새벽에 그 처리가 평소보다 오래 걸린 것이 지표에 나타났다. 배포 표시가 함께 보여 원인을 곧바로 이 배포로 좁힐 수 있었다. 새 항목으로 찾는 방식이 그 처리에서는 맞지 않았던 것이다.

이번에는 되돌리지 않고 그 처리만 옛 방식을 쓰도록 고쳤다. 영향 범위가 좁고 되돌리면 화면의 개선이 함께 사라지기 때문이다. 사흘이 지난 뒤 관찰을 끝냈다고 기록에 적었다. 관찰 기간을 처리 주기에 맞춘 것이 이 문제를 잡아 냈다. 되돌리는 대신 부분만 고친 판단도 기록에 함께 남겼다.

6.7 절차에 더한 것

이 배포에서 걸린 문제 둘은 모두 미리 물었더라면 알 수 있는 것이었다. 옛 자료에 새 항목이 채워져 있는가와, 이 항목을 쓰는 처리가 화면 말고 또 있는가다.

그래서 점검표에 두 줄을 더했다. 저장 구조를 바꾸는 배포에서만 확인하는 항목으로 두어, 다른 배포에서는 보지 않게 했다. 모든 배포에 적용되는 항목으로 만들면 대부분의 배포에서 뜻 없이 지나가기 때문이다.

배포 기록에는 되돌린 이유와 다시 컨 시점을 적었다. 몇 달 뒤 비슷한 변경을 준비하는 사람이 이 기록을 보면 같은 자리에서 멈추지 않을 수 있다. 절차는 이런 기록이 쌓여 바뀐다.조건이 있는 배포에서만 보는 항목으로 두어 목록이 길어지지 않았다.

7.1 얼마나 나눌 것인가

모든 배포를 잘게 나누면 배포가 무거워진다. 나누는 정도는 변경의 성질에 맞춘다.

기준은 둘이다. 되돌릴 수 있는가와 잘못되면 얼마나 넓게 번지는가다. 되돌릴 수 있고 영향이 좁은 변경은 한 번에 올리고, 되돌릴 수 없거나 영향이 넓은 변경은 나눈다. 둘 다 해당하면 가장 잘게 나눈다.

나누기로 했다면 각 단계가 스스로 되돌릴 수 있는 상태여야 한다. 나누기만 하고 중간 단계에서 되돌릴 수 없으면 나눈 값이 없다. 6장 사례에서 저장 구조를 먼저 올린 것도 그 단계가 혼자 되돌려질 수 있었기 때문이다. 나누는 정도를 정하는 일도 배포 준비의 일부다. 나누지 않기로 한 결정도 근거와 함께 적어 둔다. 나누는 정도를 미리 정해 두면 준비도 그만큼 가벼워진다.

7.2 되돌릴 수 있게 두기

이 책의 절차는 모두 하나로 모인다. 되돌릴 수 있는 상태를 유지하는 것이다. 점검은 되돌릴 수 없는 것을 미리 찾기 위한 것이고, 단계 나누기는 각 단계를 되돌릴 수 있게 만들기 위한 것이며, 관찰은 되돌릴지 판단하기 위한 것이다.

되돌리기는 실패가 아니라 준비된 선택이다. 되돌린 뒤 원인을 찾아 고치고 다시 올리는 것이 정상 경로이며, 이 경로가 열려 있으면 배포는 큰 결정이 아니게 된다.

반대로 되돌릴 수 없는 배포는 그 자체로 큰 결정이다. 그런 배포가 필요하면 그 사실을 분명히 하고 그에 맞는 준비를 한다. 문제는 되돌릴 수 없다는 것을 모른 채로 진행하는 경우다. 되돌릴 수 있는 배포는 결정이 아니라 절차가 된다.

7.3 점검표를 손볼 때

점검표는 고정된 문서가 아니라 계속 손보는 목록이다. 손보는 시점은 둘이다. 배포에서 문제가 걸렸을 때와 이따금 하는 정리다.

문제가 걸리면 그 문제를 미리 물었더라면 알 수 있었는지 본다. 알 수 있었다면 항목을 더하고, 알 수 없었다면 더하지 않는다. 알 수 없는 것을 항목으로 만들면 형식만 늘어난다.

정리할 때는 반대로 본다. 최근 배포에서 한 번도 걸린 적 없는 항목을 찾아 기계 확인으로 옮기거나 지운다. 조건에 따라만 필요한 항목은 그 조건이 있는 배포에서만 보게 나눈다. 짧게 유지되는 목록만 실제로 읽힌다. 목록을 손보는 일은 배포 절차를 손보는 일과 같다. 손보는 시점을 정해 두지 않으면 목록은 늘어나기만 한다.

7.4 배포 점검 항목

배포를 준비할 때는 다음을 차례로 묻는다. 첫째, 이번에 무엇이 바뀌는가. 코드·설정·저장 구조·바깥 약속 넷으로 나눠 적는다.

둘째, 되돌릴 수 있는가. 없다면 되돌릴 수 있는 형태로 나눈다. 셋째, 무엇을 보고 성공이라 하고 무엇을 보고 멈출 것인가. 넷째, 그 판단에 필요한 지표가 지금 있는가.

다섯째, 관찰을 언제까지 하고 누가 끝났다고 말하는가. 다섯 물음은 모두 배포 전에 답할 수 있고, 답하지 못하는 물음이 있으면 그 자리가 이번 배포의 위험이다. 답하지 못한 물음이 있으면 그 물음을 다른 장으로 돌아간다. 다섯 물음은 배포마다 몇 분이면 답할 수 있다. 답하지 못한 물음이 곧 이번 배포에서 볼 자리다.

7.5 관찰을 남기는 순서

관찰을 처음부터 다 갇출 수는 없으므로 순서를 둔다. 가장 먼저 배포 표시를 남긴다. 언제 무엇이 올라갔는지가 지표에 함께 보이면 원인 좁히기가 곧바로 쉬워진다.

다음은 오류와 응답 시간처럼 모든 배포에 공통으로 쓰이는 지표다. 이 둘은 한 번 갇추면 계속 쓰인다. 그다음이 배포마다 다른 지표, 곧 이번 변경이 겨논 것을 세는 값이다.

마지막이 기록이다. 관찰이 끝났다고 적는 자리와 되돌린 이유를 적는 자리를 둔다. 앞의 셋이 없으면 기록에 적을 내용이 없고, 기록이 없으면 앞의 셋이 다음 배포에 이어지지 않는다. 앞의 것부터 갇추면 뒤의 것이 훨씬 쉬워진다. 표시 하나만 있어도 조사에서 얻는 것이 크게 달라진다.

7.6 사람과 절차

배포 절차의 목적은 사람의 판단을 없애는 것이 아니라 급한 순간에 판단할 일을 줄이는 것이다. 미리 정할 수 있는 것을 미리 정해 두면 남은 판단에 집중할 수 있다.

미리 정할 수 있는 것은 기준과 순서와 결정권이다. 어떤 값을 넘으면 멈추는지, 어떤 차례로 넓히는지, 누가 결정하는지는 평온할 때 정해 두는 편이 낫다. 급할 때 정하면 그 자리의 압박이 판단에 섞인다.

반대로 절차로 정할 수 없는 것도 있다. 지금 이 변경을 올려도 되는 때인지, 되돌린 뒤 어떻게 할지는 상황을 보고 정한다. 절차는 이 판단이 잘 이루어지도록 나머지를 정리해 주는 장치다. 절차가 잘 서 있을수록 사람이 볼 것이 분명해진다. 미리 정할 수 있는 것을 미리 정해 두는 일이 절차의 전부다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 둘이다. 이번에 무엇이 바뀌는가, 그리고 잘못되면 어떻게 되돌리는가. 앞의 물음이 절차의 무게를 정하고 뒤의 물음이 나눌 자리를 정한다.

나머지는 두 물음의 도구다. 점검표는 미리 물을 것을 잊지 않게 하고, 단계 나누기는 되돌릴 수 있는 구간을 만들며, 상태 확인은 올라간 것과 도는 것을 가르고, 관찰은 멈출지 넓힐지 판단할 재료를 준다.

끝으로 덧붙일 것은 배포가 기술 작업이면서 결정 작업이라는 점이다. 무엇을 언제 올릴지, 언제 멈출지, 언제 끝났다고 할지는 모두 결정이다. 그 결정을 미리 내려 두는 만큼 배포는 조용해진다. 미리 내려 둔 결정이 많을수록 배포하는 날이 평범해진다.